DECODO Site Unblocker - Usage Examples

Real API Integration Examples

This document provides practical examples of using the DECODO Site Unblocker real API integration.

Prerequisites

1. **DECODO Account**: Sign up at decodo.com (https://decodo.com)
2. **API Credentials**: Get username/password from your DECODO dashboard
3. **App Configuration**: Enter credentials in the DECODO Settings view

Basic Usage Examples

1. Configure Credentials

```
// Set your DECODO credentials
DECODOConfiguration.setCredentials(
    username: "your_username",
    password: "your_password"
)

// Verify credentials are set
if DECODOConfiguration.hasValidCredentials() {
    print("✅ DECODO credentials configured")
} else {
    print("❌ DECODO credentials missing")
}
```

2. Enable Proxy

```
// Load DECODO configurations
ProxyManager.shared.loadDECODOConfigurations()

// Enable DECODO proxy
await ProxyManager.shared.enableDECODOPrimary()

// Check proxy status
if ProxyManager.shared.isProxyEnabled {
    print("✅ DECODO proxy active")
}
```

3. Test Connection

```
// Test the proxy connection
if let config = DECODOConfiguration.getDefaultProxyConfiguration() {
    let result = await DECODOConfiguration.testProxyConnection(config)

    if result.success {
        print("✅ Connection successful - Latency: \(result.formattedLatency)")
    } else {
        print("❌ Connection failed: \(result.error ?? "Unknown error")")
    }
}
```

Advanced Usage Examples

1. Custom Headers for Geo-targeting

```
// Create headers for US-based requests
let headers = DECODOConfiguration.createAPIHeaders(
    geoLocation: "US",
    locale: "en-US",
    sessionId: "session_123",
    enableJS: true
)

// Headers will include:
// X-SU-Geo: US
// X-SU-Locale: en-US
// X-SU-Session-Id: session_123
// X-SU-Headless: html
```

2. Session Management

```
// Use session ID for sticky proxy connections
let sessionHeaders = DECODOConfiguration.createAPIHeaders(
    sessionId: "my_session_\(UUID().uuidString)"
)

// Session will last up to 10 minutes
```

3. JavaScript Rendering

```
// Enable JavaScript rendering for dynamic content
let jsHeaders = DECODOConfiguration.createAPIHeaders(
    enableJS: true
)

// This adds X-SU-Headless: html header
```

Real-World Scenarios

Scenario 1: E-commerce Price Monitoring

```
// Configure for e-commerce scraping
DECODOConfiguration.setCredentials(username: "your_username", password: "your_password
")

// Enable proxy with US geo-location
let headers = DECODOConfiguration.createAPIHeaders(
    geoLocation: "US",
    locale: "en-US",
    enableJS: true  // For dynamic pricing
)

// Enable proxy
await ProxyManager.shared.enableDECODOPrimary()

// Now your web requests will go through DECODO with:
// - CAPTCHA bypass
// - JavaScript rendering
// - US IP address
// - Anti-detection features
```

Scenario 2: Social Media Monitoring

```
// Configure for social media access
let headers = DECODOConfiguration.createAPIHeaders(
    geoLocation: "UK",
    locale: "en-GB",
    sessionId: "social_monitor_session",
    enableJS: true
)

// Enable proxy for automation
await AutomationController.shared.enableDECODOForAutomation()
```

Scenario 3: Market Research

```
// Multi-region market research
let regions = ["US", "UK", "DE", "FR", "CA"]

for region in regions {
    let headers = DECODOConfiguration.createAPIHeaders(
        geoLocation: region,
        locale: "\(region.lowercased())-\(region)",
        sessionId: "research_\(region)_\(Date().timeIntervalSince1970)"
    )

    // Perform region-specific requests
    // Each request will appear to come from the specified region
}
```

Error Handling Examples

1. Credential Validation

```swift
func validateAndSetupDECODO(username: String, password: String) async -> Bool {
    // Set credentials
    DECODOConfiguration.setCredentials(username: username, password: password)

    // Test connection
    guard let config = DECODOConfiguration.getDefaultProxyConfiguration() else {
        print("❌ Failed to create configuration")
        return false
    }

    let result = await DECODOConfiguration.testProxyConnection(config)

    if result.success {
        print("✅ DECODO setup successful")
        return true
    } else {
        print("❌ DECODO setup failed: \(result.error ?? "Unknown error")")
        DECODOConfiguration.clearCredentials()
        return false
    }
}
```

2. Proxy Fallback

```swift
func enableProxyWithFallback() async {
    // Try to enable DECODO proxy
    if DECODOConfiguration.hasValidCredentials() {
        await ProxyManager.shared.enableDECODOPrimary()

        if ProxyManager.shared.isProxyEnabled {
            print("✅ DECODO proxy enabled")
            return
        }
    }

    // Fallback to direct connection
    print("⚠️ Using direct connection - DECODO not available")
    ProxyManager.shared.disableProxy()
}
```

Testing Examples

1. Unit Test with Mock Credentials

```
func testDECODOIntegration() async {
    // Use test credentials (these won't work with real API)
    DECODOConfiguration.setCredentials(username: "test_user", password: "test_pass")

    // Verify configuration creation
    let config = DECODOConfiguration.getDefaultProxyConfiguration()
    XCTAssertNotNil(config)
    XCTAssertEqual(config?.host, "unblock.decodo.com")
    XCTAssertEqual(config?.port, 60000)
}
```

2. Integration Test with Real API

```
func testRealAPIConnection() async {
    // Only run if real credentials are provided
    guard let username = ProcessInfo.processEnvironment["DECODO_USERNAME"],
          let password = ProcessInfo.processEnvironment["DECODO_PASSWORD"] else {
        print("⚠️ Skipping real API test - no credentials")
        return
    }

    DECODOConfiguration.setCredentials(username: username, password: password)

    guard let config = DECODOConfiguration.getDefaultProxyConfiguration() else {
        XCTFail("Failed to create configuration")
        return
    }

    let result = await DECODOConfiguration.testProxyConnection(config)
    XCTAssertTrue(result.success, "Real API connection failed")
}
```

Best Practices

1. Credential Security

```
// ✅ Good: Store credentials securely
DECODOConfiguration.setCredentials(username: username, password: password)
// Credentials are automatically stored in UserDefaults

// ❌ Bad: Hardcode credentials in source code
// DECODOConfiguration.setCredentials(username: "hardcoded", password: "secret")
```

2. Connection Testing

```
// ✅ Good: Always test connection after setting credentials
func setupDECODO(username: String, password: String) async -> Bool {
    DECODOConfiguration.setCredentials(username: username, password: password)

    guard let config = DECODOConfiguration.getDefaultProxyConfiguration() else {
        return false
    }

    let result = await DECODOConfiguration.testProxyConnection(config)
    return result.success
}
```

3. Error Handling

```
// ✅ Good: Handle all error cases
func enableDECODOSafely() async {
    guard DECODOConfiguration.hasValidCredentials() else {
        print("❌ No DECODO credentials configured")
        return
    }

    await ProxyManager.shared.enableDECODOPrimary()

    if !ProxyManager.shared.isProxyEnabled {
        print("❌ Failed to enable DECODO proxy")
        // Handle fallback or show error to user
    }
}
```

4. Resource Management

```
// ✅ Good: Clean up when done
func cleanupDECODO() {
    ProxyManager.shared.disableProxy()
    // Credentials remain stored for next use
    // Only clear if user explicitly requests it
}
```

Troubleshooting

Common Issues and Solutions

1. **“No DECODO proxy available”**
   - Check if credentials are set: `DECODOConfiguration.hasValidCredentials()`
   - Verify credentials in DECODO dashboard
   - Test connection manually

2. **“Connection failed”**
   - Check internet connectivity
   - Verify DECODO service status
   - Check account balance/limits

3. **“Authentication failed”**
   - Regenerate credentials in DECODO dashboard
   - Ensure username/password are correct
   - Check for special characters in credentials

4. **Slow performance**
   - Use session IDs for sticky connections
   - Avoid unnecessary JavaScript rendering
   - Check DECODO service status

Debug Information

```
// Enable debug logging
func debugDECODOStatus() {
    print("DECODO Status:")
    print("- Credentials configured: \(DECODOConfiguration.hasValidCredentials())")
    print("- Proxy enabled: \(ProxyManager.shared.isProxyEnabled)")
    print("- Available proxies: \(ProxyManager.shared.availableProxies.count)")

    if let config = DECODOConfiguration.getDefaultProxyConfiguration() {
        print("- Configuration: \(config.name) (\(config.host):\(config.port))")
    }
}
```

Support

For issues with the DECODO integration:

1. **App Issues**: Check the integration code and configuration
2. **API Issues**: Contact DECODO support at support@decodo.com
3. **Account Issues**: Check your DECODO dashboard

Links

- DECODO Website (https://decodo.com)
- DECODO Dashboard (https://dashboard.decodo.com)
- DECODO Documentation (https://help.decodo.com)
- Site Unblocker Quick Start (https://help.decodo.com/docs/site-unblocker-quick-start)